UNIVERSITE CHEIKH ANTA DIOP — ESP

Licence 3 — GLSi | Integration IA dans les SI

TRANSPOBOT

Auteur : Mohamed Moctar Niang

Documentation Technique & Journal de Developpement

Gestion de Transport Urbain avec IA

Pr. Ahmath Bamba MBACKE

Annee academique 2025-2026

1. Introduction et contexte du projet

Ce document constitue la documentation technique et le journal de developpement du projet TranspoBot. Il retrace l'ensemble des decisions prises, des configurations effectuees et des modifications apportees au code fourni par l'enseignant.

TranspoBot est une application web de gestion de transport urbain integrant un assistant conversationnel base sur un LLM (Large Language Model). L'objectif est de permettre aux gestionnaires d'interroger la base de donnees en langage naturel via un chatbot IA.

---

2. Configuration de l'environnement (.env)

Le fichier .env centralise toutes les variables sensibles du projet. Il ne doit jamais être pousse sur GitHub.

2.1 Contenu initial fourni par le professeur (.env.example)

Le professeur a fourni un fichier .env.example avec des valeurs generiques :

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=votre_mot_de_passe
DB_NAME=transpobot
OPENAI_API_KEY=sk-...
LLM_MODEL=gpt-4o-mini
```

2.2 Probleme rencontre — cle OpenAI sans credits

Lors des premiers tests, la cle OpenAI generait l'erreur suivante :

```
{"error": {"message": "You exceeded your current quota", "type":
"insufficient_quota"}}
```

Cause : le compte OpenAI ne disposait pas de credits suffisants (minimum 5 dollars necessaires).

2.3 Solution adoptée — Ollama

- Ollama (local gratuit) : modèle llama3 tourne en local, aucune clé requise

Solution finale retenue : Ollama avec llama3 (4.7 GB, 100% local, gratuit et sans limite).

2.4 Configuration finale du .env

Voici la configuration operationnelle du projet :

```env
.env
DB_HOST=localhost
DB_USER=transpobot_user
DB_PASSWORD=MOCTAR
DB_NAME=transpobot
OPENAI_API_KEY=ollama
LLM_MODEL=llama3
LLM_BASE_URL=http://localhost:11434/v1
```

---

3. Base de données MySQL

3.1 Importation du schéma

Le fichier schema.sql fourni par le professeur a été importe dans MariaDB via la commande

```
mysql -u root -p < schema.sql
```

Vérification de l'importation :

```
USE transpobot;
SHOW TABLES;
```

Resultat :

```
MariaDB [transpobot]> show tables;
+----------------------+
| Tables_in_transpobot |
+----------------------+
| chauffeurs           |
| conversation_logs    |
| incidents            |
| lignes               |
| tarifs               |
| trajets              |
| vehicules            |
+----------------------+
7 rows in set (0.001 sec)
```

4. Modifications apportées a app.py

4.1 Ajout du chargement du .env

Probleme initial : app.py utilisait os.getenv() mais ne chargeait jamais le fichier .env. Les variables d'environnement n'etaient donc pas lues.

Solution : ajout de load_dotenv() en haut du fichier :

```
from dotenv import load_dotenv

load_dotenv()  # charge le fichier .env
```

```
app.py > get_db
2   TranspoBot — Squelette Backend FastAPI
3   Projet GLSi L3 — ESP/UCAD
4   """
5
6   from fastapi import FastAPI, HTTPException
7   from fastapi.staticfiles import StaticFiles
8   from fastapi.middleware.cors import CORSMiddleware
9   from pydantic import BaseModel
10  import mysql.connector
11  import os
12  import re
13  import httpx
14  from fastapi.responses import StreamingResponse
15  import io
16  import csv
17  from dotenv import load_dotenv
18
19  load_dotenv()  # ← charge le fichier .env
20
21  app = FastAPI(title="TranspoBot API", version="1.0.0")
22
```

Sans cette ligne, toutes les variables (DB_PASSWORD, OPENAI_API_KEY...) restaient vides.

4.2 Sécurisation des requêtes SQL (validation)

Conformément aux exigences du cahier des charges, une fonction de validation a été ajoutée pour protéger la base de données. Le système doit accepter uniquement les requêtes SELECT et rejeter tout mot-clé destructeur.

Fonction ajoutee :

```
def validate_sql(sql: str):
    sql_upper = sql.strip().upper()
    forbidden = ['DELETE', 'DROP', 'UPDATE', 'INSERT', 'ALTER',
'GRANT', 'TRUNCATE']
    if not sql_upper.startswith('SELECT') or any(word in sql_upper
for word in forbidden):
        raise HTTPException(status_code=403, detail='Requete non
autorisee : Seuls les SELECT sont autorises.')
    return True
```

Cette fonction est appelée dans execute_query() avant toute exécution :

```
def execute_query(sql: str):
    validate_sql(sql)  # verification securite
    conn = get_db()
    ...
```

Comportement : si quelqu'un tape 'Supprime tous les véhicules' ou 'DELETE FROM vehicules', le système répond avec une erreur 403 au lieu d'exécuter la requête.

```python
# — Connexion MySQL ————————————————————————————
def get_db():
    return mysql.connector.connect(**DB_CONFIG, charset='utf8mb4', collation='utf8mb4_unicode_ci')

def validate_sql(sql: str):
    sql_upper = sql.strip().upper()
    forbidden = ["DELETE", "DROP", "UPDATE", "INSERT", "ALTER", "GRANT", "TRUNCATE"]
    if not sql_upper.startswith("SELECT") or any(word in sql_upper for word in forbidden):
        raise HTTPException(status_code=403, detail="Requête non autorisée : Seuls les SELECT sont autorisés.")
    return True

def execute_query(sql: str):
    validate_sql(sql) # Appel de la sécurité
    conn = get_db()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute(sql)
        return cursor.fetchall()
    finally:
        cursor.close()
        conn.close()
```

4.3 Correction de l'encodage UTF-8

La connexion MySQL a été mise à jour pour forcer l'encodage UTF-8 :

```
def get_db():
    return mysql.connector.connect(**DB_CONFIG, charset='utf8mb4',
collation='utf8mb4_unicode_ci')
```

4.4 Nouveaux endpoints API ajoutes

Trois nouvelles routes GET ont été implémentées pour compléter les fonctionnalités :

Endpoint	Méthode	Description
/api/incidents	GET	Incidents avec détails des trajets (JOIN)
/api/lignes	GET	Liste de toutes les lignes de transport
/api/tarifs	GET	Grille tarifaire avec nom de ligne (JOIN)

Ces routes suivent le même pattern que les routes existantes (get_vehicules, get_chauffeurs).

4.5 Ajout de la documentation Swagger

Les décorateurs FastAPI ont été utilises pour enrichir la documentation automatique accessible sur /docs :

```
@app.get("/api/incidents", tags=["Data"])
def get_incidents():
```

```
    """Retourne la liste des incidents avec les details des
trajets."""
    return execute_query("SELECT i.*, t.date_heure_depart FROM
incidents i LEFT JOIN trajets t ON i.trajet_id = t.id")
```

Le paramètre tags=["Data"] regroupe ces routes dans une section distincte dans Swagger UI.

```
186
187  @app.get("/api/incidents", tags=["Data"])
188  def get_incidents():
189      """Retourne la liste des incidents avec les détails des trajets."""
190      return execute_query("SELECT i.*, t.date_heure_depart FROM incidents i LEFT JOIN trajets t ON i.trajet_id = t.i
191
192  @app.get("/api/lignes", tags=["Data"])
193  def get_lignes():
194      """Retourne la liste des lignes de transport."""
195      return execute_query("SELECT * FROM lignes")
196
197  @app.get("/api/tarifs", tags=["Data"])
198  def get_tarifs():
199      """Retourne la grille tarifaire."""
200      return execute_query("SELECT t.*, l.nom as ligne FROM tarifs t JOIN lignes l ON t.ligne_id = l.id")
201
202  @app.get("/api/export/csv", tags=["Export"])
203  async def export_csv(sql: str):
204      """Exécute une requête et exporte le résultat en CSV."""
205      data = execute_query(sql)
206      if not data:
207          return {"message": "Aucune donnée"}
208
209      output = io.StringIO()
210      writer = csv.DictWriter(output, fieldnames=data[0].keys())
211      writer.writeheader()
212      writer.writerows(data)
```

5. Tests effectues et résultats

5.1 Tests du chatbot IA

Question posee	Statut	SQL genere
Combien de trajets cette semaine ?	OK	SELECT COUNT(*) FROM trajets WHERE DATE...
Quel chauffeur a le plus de trajets ?	OK	SELECT c.id, c.nom, c.prenom, COUNT(t.id) AS nb_trajets FROM chauffeurs c LEFT JOIN trajets t ON c.id = t.chauffeur_id GROUP BY c.id, c.nom, c.prenom ORDER BY nb_trajets DESC LIMIT 1
Quels vehicules sont en maintenance ?	OK	SELECT * FROM vehicules WHERE statut = 'maintenance'
Liste les incidents graves non resolus	OK	SELECT i.id, t.ligne_id, c.nom, c.prenom, i.type, i.description, i.date_incident FROM incidents i JOIN trajets t ON i.trajet_id = t.id JOIN chauffeurs c ON t.chauffeur_id = c.id WHERE i.gravite = 'grave' AND i.resolu = 0 LIMIT 100
Recette totale mars 2026	OK	SELECT SUM(recette) AS recette_total FROM trajets WHERE EXTRACT(MONTH FROM

Question posee	Statut	SQL genere
		date_heure_depart) = 3 AND EXTRACT(YEAR FROM date_heure_depart) = 2026
DELETE FROM vehicules (test securite)	BLOQUE	Erreur 403 - Requete non autorisee

6. Architecture technique

6.1 Flux de traitement d'une question

Voici le flux complet lorsqu'un utilisateur pose une question au chatbot :

Etape	Composant	Action
1	Frontend (index.html)	L'utilisateur tape une question et clique Envoyer
2	FastAPI (/api/chat)	Réception de la question via POST JSON
3	ask_llm()	Envoi du prompt système + question au LLM (Ollama)
4	LLM (llama3)	Génération de la requête SQL en JSON
5	validate_sql()	Vérification sécurité : SELECT uniquement
6	execute_query()	Exécution de la requête sur MySQL
7	FastAPI	Retour du résultat : {answer, sql, data, count}
8	Frontend	Affichage de la réponse + tableau de résultats

6.2 Choix technologiques

FastAPI (Python) : framework moderne, asynchrone, avec generation automatique de documentation Swagger. Idéal pour les API REST avec integration LLM.

MariaDB/MySQL : base de donnees relationnelle robuste, parfaitement adaptee aux donnees structurees de transport.

Ollama + llama3 : solution locale gratuite, aucune dependance externe, aucun cout. Compatible avec l'interface OpenAI.

HTML/CSS/JS vanilla : frontend leger sans framework, facile a maintenir et a deployer.

6.3 Strategie de securite

La securite repose sur un pare-feu applicatif en deux niveaux :

- Niveau 1 - Prompt systeme : le LLM est instruite de ne générer que des SELECT via les instructions du SYSTEM_PROMPT.
- Niveau 2 - Validation code : la fonction validate_sql() vérifie indépendamment que la requête commence par SELECT et ne contient aucun mot-clé destructeur.
- Niveau 3 - Principe du moindre privilège : au niveau du SGBD (Système de Gestion de Base de Données) en créant un utilisateur dédié avec des droits restreints

Cette triple protection garantit qu'aucune donnée ne peut être modifiée ou supprimée via le chatbot, même si le LLM génère une requête malveillante.

7. Difficultés rencontrées et solutions

Probleme	Cause	Solution
Le chat repondait 'undefined'	URL API incorrecte dans index.html (/docs au lieu de la racine)	Corriger const API = 'http://localhost:8000'
Erreur 500 sur /api/chat	load_dotenv() manquant dans app.py	Ajouter from dotenv import load_dotenv et load_dotenv()
Cle OpenAI insufficient_quota	Compte sans credits (5$ minimum requis)	Migration vers Ollama (llama3 local, gratuit)
ollama non reconnu dans VSCode	PATH non mis a jour apres installation	Fermer/rouvrir VSCode ou utiliser cmd Windows
Caracteres speciaux casses (Thies)	Base creee en latin1 avant config utf8mb4	ALTER TABLE ... CONVERT TO CHARACTER SET utf8mb4

---

8. Conclusion et prochaines étapes

L'application TranspoBot est fonctionnelle en local. Le tableau de bord affiche les KPIs en temps reel, le chatbot IA genere et execute des requetes SQL a partir de questions en langage naturel, et le systeme de securite bloque toute tentative de modification de la base.